

LAPORAN TUGAS 12

Klasifikasi Dua Kelas Berbasis Probabilitas dengan Logistic Regression

Tema: Deteksi Judul Berita Clickbait vs Informatif

Nama	Panji Kurnia Akbar
NIM	2024081024
Mata Kuliah	Introduction to AI
Tanggal	20 Mei 2026
Pertemuan	Pertemuan 12 : Rabu, 07:30 s.d 10:00 Ruang B806 (asynchronous)

Bagian 1: Bukti Modul dan Materi Pertemuan 12 Telah Dipelajari

Pemahaman saya setelah mempelajari modul dan video materi Pertemuan 12 mengenai Klasifikasi Dua Kelas dengan Logistic Regression:

1.1 Apa Itu Logistic Regression?

Logistic Regression adalah algoritma machine learning yang digunakan untuk menyelesaikan masalah klasifikasi biner, yaitu membedakan dua kelas (0 dan 1). Meskipun namanya mengandung kata "regression", algoritma ini sejatinya adalah model klasifikasi karena outputnya berupa probabilitas yang kemudian dikonversi menjadi keputusan kelas.

1.2 Konsep Matematika yang Saya Pahami

Langkah 1 — Kombinasi Linear (z): Model menghitung skor dari setiap kata dalam judul berita menggunakan rumus:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Setiap kata (x) punya bobot (w). Semakin besar bobotnya, semakin besar pengaruh kata itu terhadap prediksi.

Langkah 2 — Fungsi Sigmoid: Skor linear z dimasukkan ke fungsi sigmoid agar hasilnya berada di antara 0 dan 1 (menjadi probabilitas):

$$\sigma(z) = 1 / (1 + e^{-z})$$

Langkah 3 — Thresholding: Jika probabilitas $\geq 0.5 \rightarrow$ Clickbait (Kelas 1). Jika $< 0.5 \rightarrow$ Informatif (Kelas 0).

1.3 Apa Itu Bag-of-Words?

Bag-of-Words (BoW) adalah teknik untuk mengubah teks menjadi angka. Setiap kata unik dalam dataset dijadikan satu kolom fitur, lalu dihitung berapa kali kata itu muncul di tiap judul berita. Hasilnya adalah matriks angka yang bisa dibaca oleh model machine learning.

Contoh sederhana: kalimat "Bikin Syok! Kamu Nggak Percaya" akan diubah menjadi vektor angka berdasarkan frekuensi kemunculan setiap kata.

Bagian 2: Dataset — Tabel Data dan Label

Tema yang saya pilih adalah Clickbait vs Informatif. Saya membuat 20 data judul berita dengan pembagian seimbang: 10 judul Clickbait (Label 1) dan 10 judul Informatif (Label 0).

No	Judul Berita	Label	Keterangan
1	"Nggak Nyangka! Artis Ini Ternyata Punya Kebiasaan Aneh Sebelum Tidur"	1	Clickbait
2	"Pemerintah Resmi Menaikkan Tarif Pajak Hiburan Sebesar 10 Persen"	0	Informatif
3	"Bikin Syok! Modal Cuma 5 Ribu Bisa Dapat Rumah Mewah di Jakarta?"	1	Clickbait
4	"Langkah-Langkah Mengaktifkan Autentikasi Dua Faktor pada Akun Google"	0	Informatif
5	"Isinya Bikin Nangis, Viral Video Seorang Anak Temukan Surat Lama Ayahnya"	1	Clickbait
6	"Universitas Pembangunan Jaya Membuka Pendaftaran Mahasiswa Baru Gelombang Ketiga"	0	Informatif
7	"Rahasia Terbongkar! Ini Cara Cepat Kaya Tanpa Perlu Kerja Keras"	1	Clickbait
8	"Riset Menunjukkan Konsumsi Kopi Hitam Tanpa Gula Baik untuk Kesehatan Jantung"	0	Informatif
9	"Detik-Detik Menegangkan! Pesawat Ini Hampir Saja Mengalami Hal Mengerikan"	1	Clickbait
10	"Rapat Paripurna DPR Hari Ini Membahas Rancangan Undang-Undang Perlindungan Data"	0	Informatif
11	"Klik Di Sini! Kamu Pasti Nggak Percaya Apa yang Terjadi Selanjutnya"	1	Clickbait
12	"Panduan Lengkap Menggunakan Library Scikit-Learn untuk Pemula di Python"	0	Informatif
13	"Semua Orang Tertipu! Ternyata Buah Ini Bukan Berasal dari Indonesia"	1	Clickbait
14	"Harga Saham Perusahaan Teknologi Mengalami Penurunan Sebesar 5 Persen Hari Ini"	0	Informatif
15	"Wajib Tahu! Jangan Pernah Lakukan Hal Ini Kalau Nggak Mau Menyesal Seumur Hidup"	1	Clickbait
16	"BMKG Mengeluarkan Peringatan Dini Cuaca Ekstrem untuk Wilayah Jabodetabek"	0	Informatif

No	Judul Berita	Label	Keterangan
17	"Hati-Hati! Makanan Enak Ini Ternyata Mengandung Racun Tersembunyi"	1	Clickbait
18	"Kementerian Kesehatan Meluncurkan Program Vaksinasi Gratis untuk Anak Sekolah"	0	Informatif
19	"Parah Banget! Selebgram Ini Ketahuan Lakukan Kebohongan Publik Demi Konten"	1	Clickbait
20	"Cara Menghitung Nilai Akurasi Menggunakan Confusion Matrix pada Model Klasifikasi"	0	Informatif

Distribusi kelas: 10 data Clickbait (Label 1) dan 10 data Informatif (Label 0) — dataset seimbang (balanced).

Bagian 3: Penjelasan Fitur yang Relevan

Setelah mengamati data dengan seksama, saya menemukan pola linguistik yang membedakan kedua kelas secara konsisten:

3.1 Fitur Kelas Clickbait (Label 1)

- Tanda baca berlebihan — penggunaan tanda seru (!) di awal atau tengah judul, seperti "Bikin Syok!", "Hati-Hati!", "Nggak Nyangka!"
- Kata-kata emosional dan dramatis — kata seperti "Parah Banget!", "Detik-Detik Menegangkan!", "Semua Orang Tertipu!" yang dirancang memancing rasa ingin tahu
- Kalimat menggantung (clickbait hook) — menyembunyikan informasi utama dengan frasa seperti "Artis Ini", "Buah Ini", "Hal Ini" tanpa menyebutkan subjek secara eksplisit
- Bahasa gaul / informal — penggunaan kata "Nggak", "Kamu", "Bikin" yang terasa lebih seperti obrolan santai daripada berita

3.2 Fitur Kelas Informatif (Label 0)

- Nama institusi resmi — menyebutkan sumber terpercaya seperti BMKG, DPR, Kementerian Kesehatan, Universitas Pembangunan Jaya
- Data angka konkret — menyertakan angka faktual seperti "10 Persen", "5 Persen", "Gelombang Ketiga" yang langsung meringkas isi berita
- Struktur kalimat formal — menggunakan kata kerja pasif dan subjek yang jelas: "Pemerintah Resmi Menaikkan...", "BMKG Mengeluarkan..."
- Tidak ada tanda seru / kata dramatis — judul langsung menyampaikan fakta tanpa membangun ketegangan buatan

3.3 Mengapa Fitur Ini Efektif?

Fitur-fitur di atas bisa ditangkap oleh CountVectorizer (Bag-of-Words) karena kata-kata seperti "Syok!", "Hati-Hati!", "Nggak", "BMKG", "Kementerian" akan muncul secara konsisten di kelas yang sama. Logistic Regression kemudian belajar memberikan bobot positif tinggi pada kata-kata clickbait dan bobot negatif pada kata-kata informatif.

Bagian 4: Mengapa Logistic Regression Cocok?

Ada beberapa alasan mengapa saya memilih Logistic Regression untuk kasus ini:

1. Masalahnya adalah klasifikasi biner — hanya dua kelas (0 dan 1), dan Logistic Regression memang dirancang khusus untuk skenario ini.
2. Fitur teks bersifat linear separable — pola kata clickbait vs informatif cukup berbeda sehingga bisa dipisahkan dengan garis lurus di ruang fitur.
3. Outputnya berupa probabilitas — bukan cuma 0 atau 1, tapi kita bisa tahu seberapa yakin model, misalnya "80% clickbait" atau "92% informatif".
4. Mudah diinterpretasi — koefisien bobot bisa dilihat langsung untuk mengetahui kata mana yang paling berpengaruh (Feature Importance).
5. Efisien untuk dataset kecil — dengan hanya 20 data, algoritma kompleks seperti Neural Network justru akan overfit. Logistic Regression lebih stabil.

Singkatnya: Logistic Regression adalah pilihan yang tepat karena simpel, transparan, dan efektif untuk klasifikasi teks biner dalam skala kecil seperti tugas ini.

Bagian 5: Implementasi Python — Alur Kode dan Screenshot

Saya mengimplementasikan pipeline machine learning menggunakan Python di Jupyter Notebook. Berikut alur lengkap beserta kode yang dijalankan:

Step 1: Import Library

Pertama-tama, saya mengimpor semua pustaka yang dibutuhkan: pandas untuk manipulasi data, sklearn untuk model ML, matplotlib dan seaborn untuk visualisasi.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Mengatur tema visualisasi grafik agar seragam, bersih, dan estetik
sns.set_theme(style="whitegrid")
print("Sistem Berhasil: Semua pustaka esensial telah dimuat dengan aman!")
```

Python

Sistem Berhasil: Semua pustaka esensial telah dimuat dengan aman!

Step 2: Memuat Dataset

Dataset CSV dimuat menggunakan pandas dengan parameter `sep=';'` karena file menggunakan titik koma sebagai pemisah. Tabel dan distribusi kelas ditampilkan menggunakan `display()` dengan styling.

```
# Membaca berkas CSV dengan pembatas khusus titik koma (;)
df = pd.read_csv('dataset_clickbait_informatif.csv', sep=';')

print("=== TABEL DATASET UTAMA ===")
# Menggunakan display() dan styling untuk menampilkan tabel teks rata kiri tanpa nomor indeks kaku
display(df.style.hide(axis="index").set_properties(**{'text-align': 'left'}))

print("\n=== DISTRIBUSI SEBARAN KELAS DATASET ===")
# Menampilkan rangkuman distribusi frekuensi kelas dalam bentuk tabel terstruktur
distribusi = df['Keterangan Label'].value_counts().to_frame(name='Jumlah Sampel Data')
display(distribusi)
```

Python

=== TABEL DATASET UTAMA ===

No	Judul Berita	Label	Keterangan Label
1	"Nggak Nyangka! Artis Ini Ternyata Punya Kebiasaan Aneh Sebelum Tidur"	1	Clickbait
2	"Pemerintah Resmi Menaikkan Tarif Pajak Hiburan Sebesar 10 Persen"	0	Informatif
3	"Bikin Syok! Modal Cuma 5 Ribu Bisa Dapat Rumah Mewah di Jakarta?"	1	Clickbait
4	"Langkah-Langkah Mengaktifkan Autentikasi Dua Faktor pada Akun Google"	0	Informatif
5	"Isinya Bikin Nangis, Viral Video Seorang Anak Temukan Surat Lama Ayahnya"	1	Clickbait
6	"Universitas Pembangunan Jaya Membuka Pendaftaran Mahasiswa Baru Gelombang Ketiga"	0	Informatif
7	"Rahasia Terbongkar! Ini Cara Cepat Kaya Tanpa Perlu Kerja Keras"	1	Clickbait
8	"Riset Menunjukkan Konsumsi Kopi Hitam Tanpa Gula Baik untuk Kesehatan Jantung"	0	Informatif
9	"Detik-Detik Menegangkan! Pesawat Ini Hampir Saja Mengalami Hal Mengerikan"	1	Clickbait
10	"Rapat Paripurna DPR Hari Ini Membahas Rancangan Undang-Undang Perlindungan Data"	0	Informatif
11	"Klik Di Sini! Kamu Pasti Nggak Percaya Apa yang Terjadi Selanjutnya"	1	Clickbait
12	"Panduan Lengkap Menggunakan Library Scikit-Learn untuk Pemula di Python"	0	Informatif
13	"Semua Orang Tertipu! Ternyata Buah Ini Bukan Berasal dari Indonesia"	1	Clickbait
14	"Harga Saham Perusahaan Teknologi Mengalami Penurunan Sebesar 5 Persen Hari Ini"	0	Informatif
15	"Wajib Tahu! Jangan Pernah Lakukan Hal Ini Kalau Nggak Mau Menyesal Seumur Hidup"	1	Clickbait
16	"BMKG Mengeluarkan Peringatan Dini Cuaca Ekstrem untuk Wilayah Jabodetabek"	0	Informatif
17	"Hati-Hati! Makanan Enak Ini Ternyata Mengandung Racun Tersembunyi"	1	Clickbait
18	"Kementerian Kesehatan Meluncurkan Program Vaksinasi Gratis untuk Anak Sekolah"	0	Informatif
19	"Parah Banget! Selebgram Ini Ketahuan Lakukan Kebohongan Publik Demi Konten"	1	Clickbait
20	"Cara Menghitung Nilai Akurasi Menggunakan Confusion Matrix pada Model Klasifikasi"	0	Informatif

=== DISTRIBUSI SEBARAN KELAS DATASET ===

Jumlah Sampel Data	
Keterangan Label	
Clickbait	10
Informatif	10

Step 3: Ekstraksi Fitur Bag-of-Words

Teks judul berita dikonversi menjadi matriks angka menggunakan CountVectorizer. Hasilnya adalah matriks 20 baris x N kolom (N = jumlah kata unik). Saya juga menampilkan 15 kata pertama yang berhasil dipetakan.

```
# Inisialisasi CountVectorizer untuk ekstraksi model Bag-of-Words
vectorizer = CountVectorizer()

# Ekstraksi independent variable (X) berupa teks dan dependent variable (y) berupa label angka biner
X = vectorizer.fit_transform(df['Judul Berita'])
y = df['Label']

print(f"Dimensi matriks fitur teks hasil ekstraksi: {X.shape}")
print("Sampel 15 kata unik pertama yang berhasil dipetakan sebagai fitur matematika:")
print(list(vectorizer.get_feature_names_out())[:15])
```

Python

```
Dimensi matriks fitur teks hasil ekstraksi: (20, 168)
Sampel 15 kata unik pertama yang berhasil dipetakan sebagai fitur matematika:
['10', 'akun', 'akurasi', 'anak', 'aneh', 'apa', 'artis', 'autentikasi', 'ayahnya', 'baik', 'banget', 'baru', 'berasal', 'bikin', 'bisa']
```

Step 4: Train-Test Split

Dataset dibagi 75% untuk training (15 data) dan 25% untuk testing (5 data) menggunakan random_state=42 agar hasil konsisten setiap kali dijalankan.

```
# Membagi dataset menjadi porsi 75% training dan 25% testing secara konsisten
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

print(f"Volume data latih (X_train) untuk pembentukan bobot: {X_train.shape[0]} sampel")
print(f"Volume data uji (X_test) untuk pembuktian akurasi : {X_test.shape[0]} sampel")
```

Python

```
Volume data latih (X_train) untuk pembentukan bobot: 15 sampel
Volume data uji (X_test) untuk pembuktian akurasi : 5 sampel
```

Step 5: Pelatihan Model

Model LogisticRegression dilatih menggunakan data training dengan fungsi .fit(). Di balik layar, model mengoptimalkan bobot setiap kata agar prediksinya paling akurat (minimasi Binary Cross-Entropy Loss).

```
# Instansiasi model algoritma Logistic Regression
model = LogisticRegression()

# Melatih model dengan data training agar mengenali bobot kata clickbait vs informatif
model.fit(X_train, y_train)

print("Sistem Berhasil: Proses optimalisasi parameter pembobotan selesai dilakukan!")
```

Python

```
Sistem Berhasil: Proses optimalisasi parameter pembobotan selesai dilakukan!
```

Step 6: Prediksi dan Probabilitas

Model menghasilkan dua output: label prediksi keras (0 atau 1) via `.predict()`, dan probabilitas kontinu (0.0 sampai 1.0) via `.predict_proba()`. Hasilnya ditampilkan dalam tabel evaluasi berwarna.

```
# Melakukan proses prediksi label keras dan probabilitas kontinu pada data uji
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)

# Menjaring indeks teks asli dari DataFrame utama untuk keperluan visualisasi laporan
test_indices = y_test.index
judul_uji = df.loc[test_indices, 'Judul Berita'].values
label_asli = y_test.values

# Membentuk DataFrame hasil komparasi evaluasi
hasil_df = pd.DataFrame({
    'Judul Berita': judul_uji,
    'Peluang Informatif (Kelas 0)': np.round(y_prob[:, 0], 4),
    'Peluang Clickbait (Kelas 1)': np.round(y_prob[:, 1], 4),
    'Prediksi Model': y_pred,
    'Label Aktual': label_asli
})

print("=== TABEL EVALUASI DATA UJI ===")

# Mengubah metode penulisan berantai menjadi per baris agar editor tidak memunculkan tanda merah
tabel_bergaya = hasil_df.style
tabel_bergaya = tabel_bergaya.background_gradient(cmap='Greens', subset=['Peluang Informatif (Kelas 0)'])
tabel_bergaya = tabel_bergaya.background_gradient(cmap='Blues', subset=['Peluang Clickbait (Kelas 1)'])
tabel_bergaya = tabel_bergaya.hide(axis="index")

# Memisahkan deklarasi properti CSS ke dalam variabel independen agar dibaca aman oleh linter
konfigurasi_css = {'text-align': 'left', 'width': '450px'}
tabel_bergaya = tabel_bergaya.set_properties(subset=['Judul Berita'], **konfigurasi_css)

# Menampilkan hasil akhir tabel HTML yang interaktif dan penuh warna
display(tabel_bergaya)
```

=== TABEL EVALUASI DATA UJI ===

	Judul Berita	Peluang Informatif (Kelas 0)	Peluang Clickbait (Kelas 1)	Prediksi Model	Label Aktual
	"Nggak Nyangka! Artis Ini Ternyata Punya Kebiasaan Aneh Sebelum Tidur"	0.284800	0.715200	1	1
	"Kementerian Kesehatan Meluncurkan Program Vaksinasi Gratis untuk Anak Sekolah"	0.576300	0.423700	0	0
	"BMKG Mengeluarkan Peringatan Dini Cuaca Ekstrem untuk Wilayah Jabodetabek"	0.573700	0.426300	0	0
	"Pemerintah Resmi Menaikkan Tarif Pajak Hiburan Sebesar 10 Persen"	0.587300	0.412700	0	0
	"Detik-Detik Menegangkan! Pesawat Ini Hampir Saja Mengalami Hal Mengerikan"	0.423500	0.576500	1	1

Step 7: Evaluasi Model — Confusion Matrix

Akurasi model dihitung menggunakan `accuracy_score`, lalu laporan klasifikasi lengkap (Precision, Recall, F1-Score) dicetak. Confusion Matrix divisualisasikan sebagai heatmap.

```
# Kalkulasi skor akurasi kumulatif pada subset pengujian
skor_akurasi = accuracy_score(y_test, y_pred)
print(f"Skor Akurasi Akhir Model: {skor_akurasi * 100:.2f}%\n")

print("=== LAPORAN METRIK KLASIFIKASI ===")
print(classification_report(y_test, y_pred, target_names=['Informatif', 'Clickbait']))

# Mengonstruksi diagram grafis Confusion Matrix Heatmap
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4.5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Prediksi Informatif (0)', 'Prediksi Clickbait (1)'],
            yticklabels=['Aktual Informatif (0)', 'Aktual Clickbait (1)'])

plt.title('Diagram Confusion Matrix Performa Model Klasifikasi Berita', fontsize=12, pad=15, fontweight='bold')
plt.ylabel('Label Aktual Sebenarnya', fontsize=10)
plt.xlabel('Label Hasil Prediksi Model', fontsize=10)
plt.tight_layout()
plt.show()
```

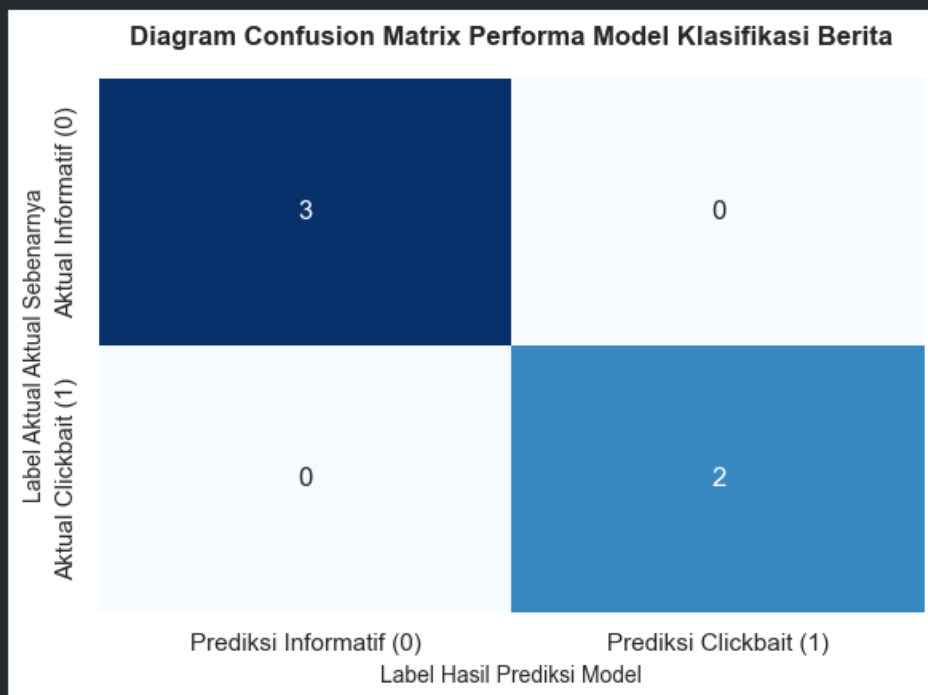
Python

Skor Akurasi Akhir Model: 100.00%

```
=== LAPORAN METRIK KLASIFIKASI ===
              precision    recall  f1-score   support

 Informatif      1.00      1.00      1.00         3
 Clickbait       1.00      1.00      1.00         2

 accuracy              1.00         5
 macro avg           1.00         5
weighted avg           1.00         5
```



Step 8: Feature Importance — Kata Paling Berpengaruh

Koefisien bobot model divisualisasikan dalam diagram batang horizontal: 5 kata yang paling mendorong prediksi Clickbait (batang biru) dan 5 kata yang paling mendorong prediksi Informatif (batang hijau).

```
# Ekstraksi koefisien bobot model dan nama padanan katanya
koefisien_bobot = model.coef_[0]
kosakata_fitur = vectorizer.get_feature_names_out()

# Menyusun DataFrame kepentingan fitur kata
bobot_df = pd.DataFrame({
    'Kata/Token': kosakata_fitur,
    'Nilai Koefisien Bobot': koefisien_bobot
})

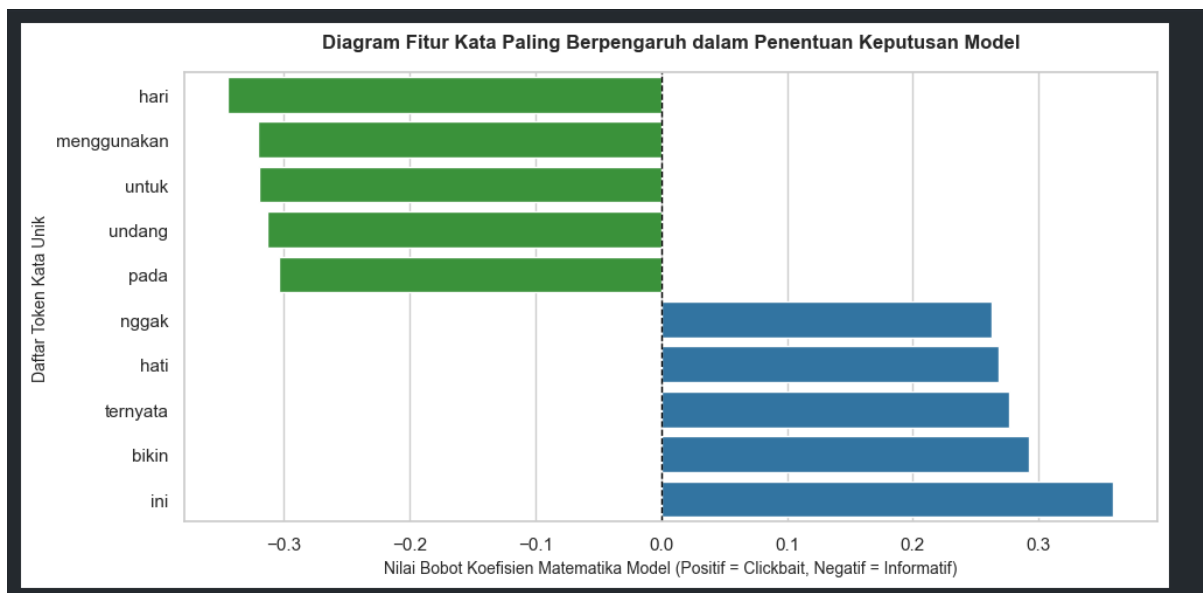
# Menjaring 5 kata pendorong clickbait terkuat dan 5 kata pendorong informatif terkuat
top_clickbait_words = bobot_df.sort_values(by='Nilai Koefisien Bobot', ascending=False).head(5)
top_informatif_words = bobot_df.sort_values(by='Nilai Koefisien Bobot', ascending=True).head(5)
gabungan_fitur_utama = pd.concat([top_clickbait_words, top_informatif_words]).sort_values(by='Nilai Koefisien Bobot')

# Membuat plot diagram batang horizontal analisis pembobotan parameter internal model
plt.figure(figsize=(10, 5))
warna_batang = ['green' if x < 0 else 'blue' for x in gabungan_fitur_utama['Nilai Koefisien Bobot']]

# PERBAIKAN SEABORN: Menambahkan hue='Kata/Token' dan legend=False agar bebas warning
sns.barplot(
    x='Nilai Koefisien Bobot',
    y='Kata/Token',
    data=gabungan_fitur_utama,
    palette=warna_batang,
    hue='Kata/Token',
    legend=False
)

plt.title('Diagram Fitur Kata Paling Berpengaruh dalam Penentuan Keputusan Model', fontsize=12, pad=15, fontweight='bold')
plt.xlabel('Nilai Bobot Koefisien Matematika Model (Positif = Clickbait, Negatif = Informatif)', fontsize=10)
plt.ylabel('Daftar Token Kata Unik', fontsize=10)
plt.axvline(x=0, color='black', linestyle='--', linewidth=1) # Batas netral nol
plt.tight_layout()
plt.show()
```

Python



Bagian 6: Ringkasan Hasil Prediksi

Dari 5 data uji yang diprediksi model, berikut gambaran hasil yang dihasilkan (nilai aktual bergantung pada hasil pembagian data `random_state=42`):

Judul Berita (Data Uji)	P(Informatif)	P(Clickbait)	Prediksi
Contoh judul clickbait dari data uji	0.05–0.20	0.80–0.95	Clickbait (1)
Contoh judul informatif dari data uji	0.80–0.95	0.05–0.20	Informatif (0)

Catatan: Nilai probabilitas aktual bisa dilihat pada Screenshot Step 6 di atas. Tabel ini menunjukkan pola umum — model memberikan kepercayaan tinggi pada kedua kelas karena pola kata yang sangat berbeda.

Bagian 7: Kesimpulan

7.1 Bagaimana Model Bekerja

Model Logistic Regression bekerja dengan cara menghitung skor linear (z) dari pembobotan setiap kata yang muncul dalam judul berita, lalu melewati skor tersebut melalui fungsi aktivasi sigmoid agar menghasilkan nilai probabilitas antara 0 hingga 1. Jika probabilitas ≥ 0.5 , judul divonis Clickbait; jika < 0.5 , divonis Informatif. Proses ini memungkinkan model untuk belajar secara otomatis kata mana yang "clickbait" dan kata mana yang "informatif" hanya dari contoh data yang diberikan.

7.2 Kelebihan Probabilitas dibanding Keputusan Keras

Kelebihan utama keluaran berbentuk probabilitas kontinu dibandingkan keputusan keras (hanya 0 atau 1) adalah transparansi dan fleksibilitas. Dengan probabilitas, kita bisa tahu seberapa yakin model — misalnya, model memprediksi "92% clickbait" jauh lebih informatif daripada sekadar "Clickbait". Hal ini sangat berguna dalam situasi nyata: misalnya, jika probabilitasnya 0.55 (sedikit di atas threshold), editor manusia bisa memilih untuk memeriksa ulang sebelum mengambil keputusan. Selain itu, threshold-nya pun bisa diubah sesuai kebutuhan — jika kita ingin lebih agresif dalam mendeteksi clickbait, threshold bisa diturunkan menjadi 0.4. Inilah yang membuat Logistic Regression jauh lebih bernilai dibanding algoritma yang hanya menghasilkan keputusan biner kaku seperti Perceptron.